



# Telegram\_Restricted\_Media\_Downloader

A telegram downloader on windows and linux platform based on Python.

Python 3.13.2

pyrogram@kurigram 2.2.17

Platform Windows & Linux

## Note

由于本项目提供的Linux版本可能对较早版本的Linux系统兼容性较差。  
若无法运行的Linux用户请阅读:[3.0.在生产环境中运行\(对于Linux用户\)](#)。  
如果你遇到任何问题, 请先仔细阅读:[常见问题及解决方案汇总](#)。  
没有找到解决方案再进群或私聊提问。

## 免责声明:

本项目以 MIT 协议开源发布, 本项目及依赖本项目发布的软件均为免费非盈利性质。仅限于合法、合规的用途。严禁使用本软件从事任何违反法律法规、侵犯他人合法权益或干扰平台正常运营的行为。

**所有使用本软件的行为及其后果均由使用者自行承担全部法律责任, 开发者不对任何使用行为及其后果负责。**

本项目按“原样”提供, 不附带任何明示或暗示的保证。

作者:[Gentlesprite](#)

B站视频教程:[点击观看](#)

Telegram交流群:[点击加入](#)

## 1.0.下载地址:

| 平台          | 下载地址                 | 备注      |
|-------------|----------------------|---------|
| 蓝奏云         | <a href="#">点击跳转</a> | 密码:ceze |
| Github      | <a href="#">点击跳转</a> | —       |
| Gitcode     | <a href="#">点击跳转</a> | —       |
| Gitee       | <a href="#">点击跳转</a> | 仅发布源码   |
| Telegram交流群 | <a href="#">点击加入</a> | 群文件     |

## 1.1.(选看)推荐终端:

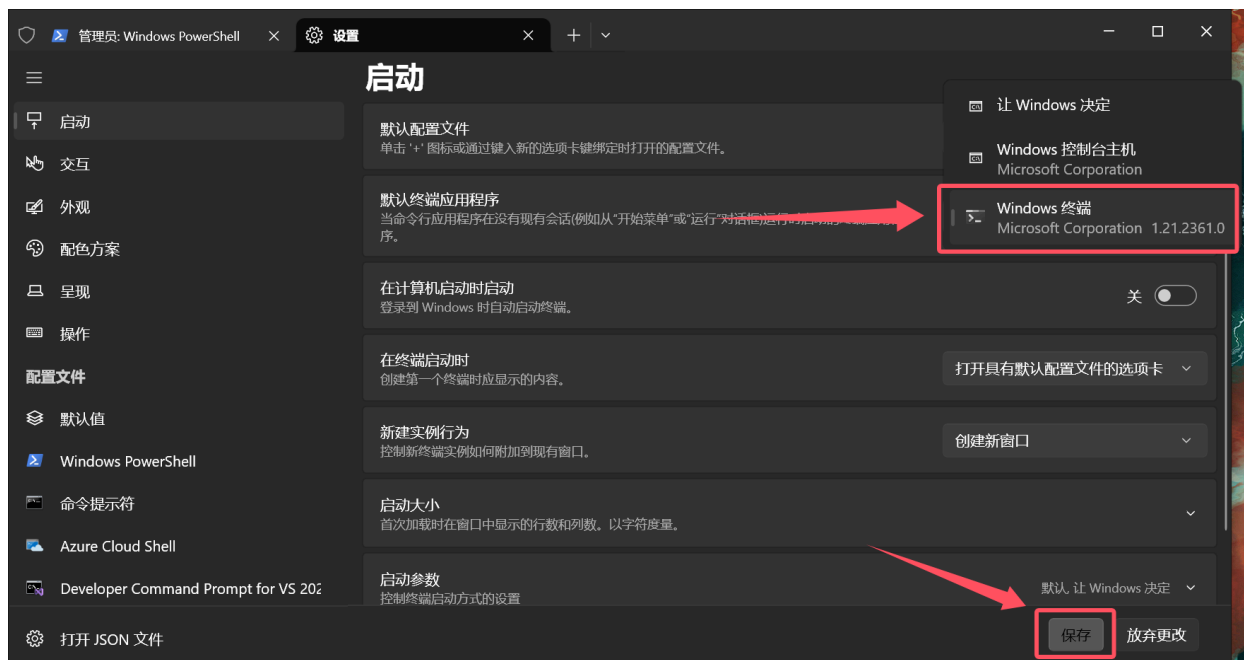
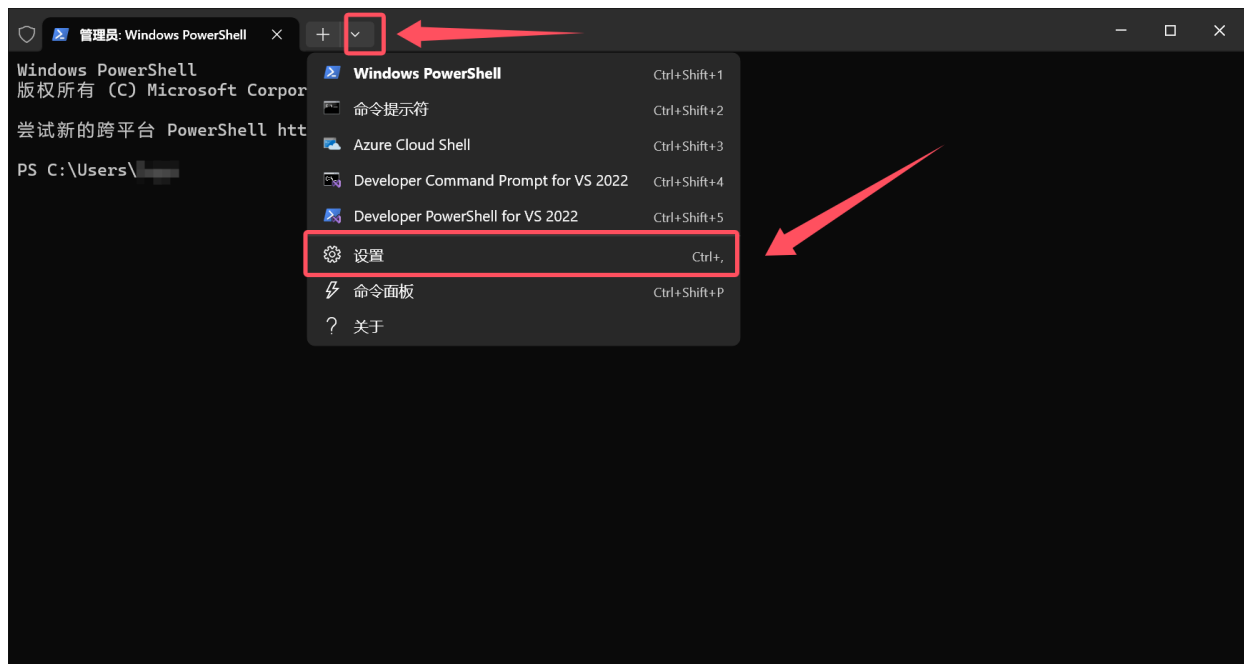
### ▼ 点击展开

1. 对于Windows11用户, Windows Terminal 默认**已经安装好**, 可**跳过**下载的步骤, 直接前往第3步。(将 Windows Terminal 设为**默认终端**)
2. 对于Windows10用户, 推荐使用 Windows Terminal 作为**默认终端**, 仅作为推荐安装, 无论安装与否**不会影响本软件的使用**, Windows Terminal 能提供更出色的显示、交互、体验效果, 以及避免出现**文字显示乱码**。

Windows Terminal 下载地址如下表:

| 平台     | 下载地址                 |
|--------|----------------------|
| 微软商店   | <a href="#">点击跳转</a> |
| Github | <a href="#">点击跳转</a> |

3. 下载完成完成后 win+r 输入 wt 回车打开, 然后将 Windows Terminal 设为**默认终端**再启动软件, 教程如下:



## 2.0.快速开始:

### 2.1.申请电报API:

1. 前往网站:<https://my.telegram.org/auth>
2. 填写**自己绑定** Telegram 电报的**手机号**注意手机号格式先要+地区再写入电话号码例如 +12223334455 , +1 为地区, 222333445 为你绑定 Telegram 的**手机号**, 填写后点击 Next 。



## Delete Account or Manage Apps

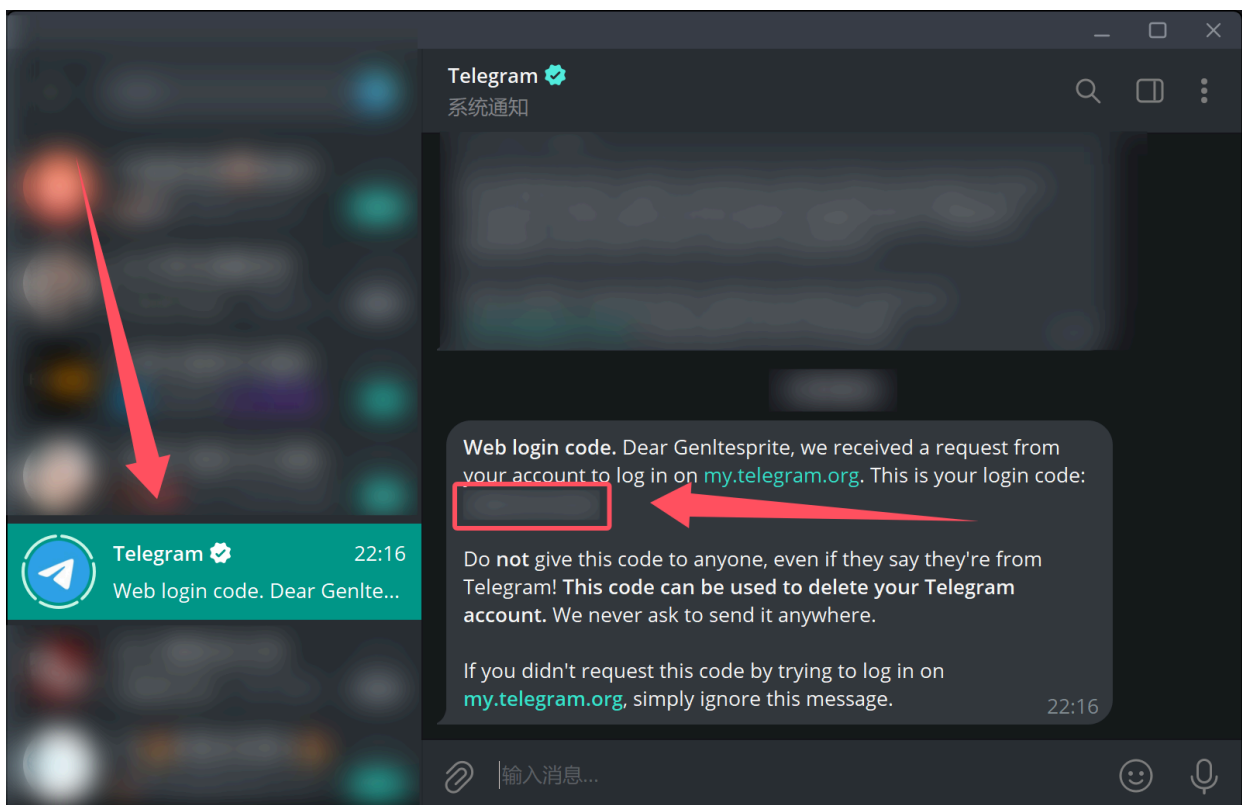
Log in here to **manage your apps** using Telegram API or **delete your account**. Enter your number and we will send you a confirmation code via Telegram (not SMS).

**Your Phone Number**

Please enter your number in **international format**

**Next**

3. 打开你的 Telegram 客户端，此时会收到来自 Telegram 账号的消息，将上面的验证码填入 Confirmation code 框中，然后点击 Sign in。





## Delete Account or Manage Apps

Log in here to **manage your apps** using Telegram API or **delete your account**. Enter your number and we will send you a confirmation code via Telegram (not SMS).

**Your Phone Number**

**Confirmation code**

☐ Remember Me

[Sign In](#)

4. 点击 API development tools 按照提示填入即可。



## Your Telegram Core

- [API development tools](#)
- [Delete account](#)
- [Log out](#)

5. 申请成功会得到一个 api\_hash 和 api\_id 保存下载，切记不要泄露给任何人！

## 2.2.(选看)电报机器人(bot\_token)申请及使用教程:

 **Note**

如果配置了机器人，只要**保持软件运行**，就能实现**多端发送下载命令并且随时进行下载**。

故可以将软件部署在服务器上，无论是Windows还是Linux平台。

Windows平台可直接使用[releases](#)里发布的二进制文件放在服务器运行。

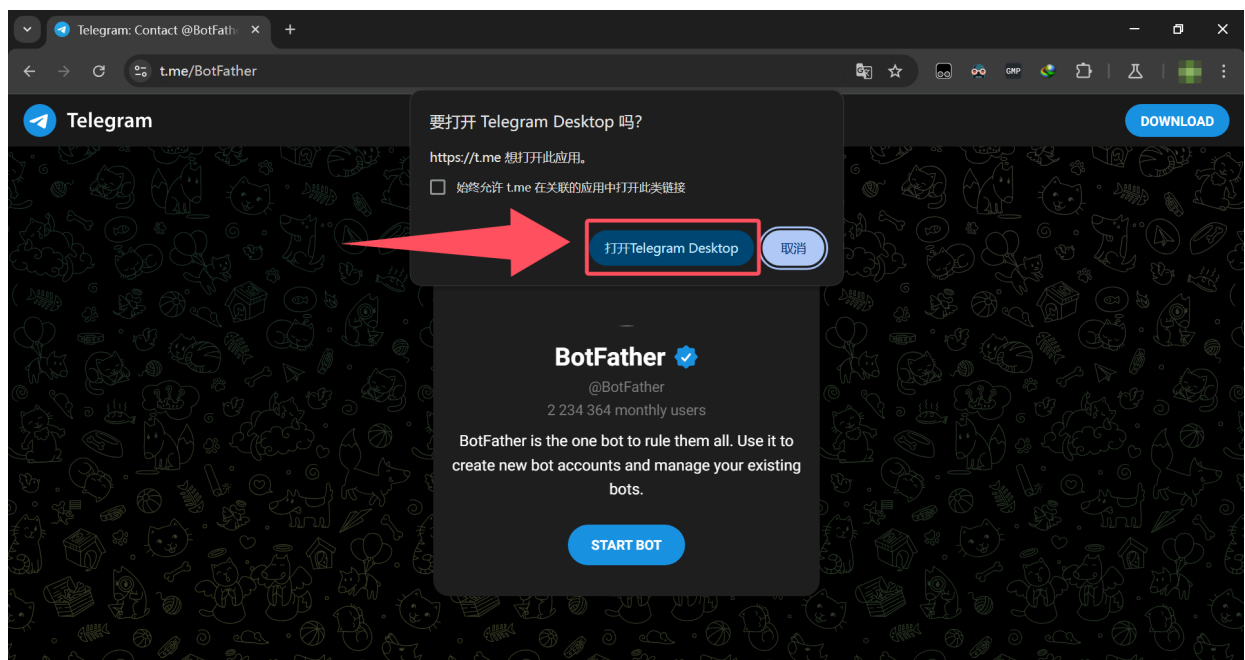
Linux平台的部署教程请阅读：["3.0.在生产环境中运行\(对于Linux用户\)"](#)。

## ▼ 点击展开

### 2.2.1.申请教程:

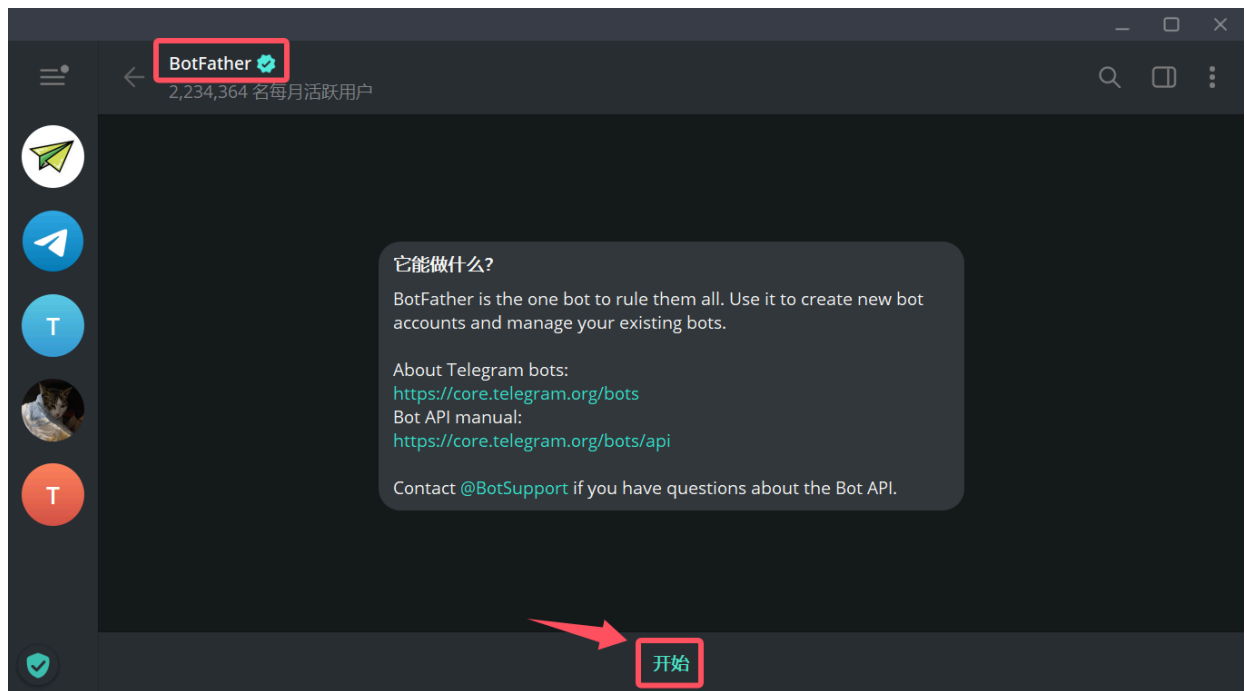
1. 前往网站:<https://t.me/BotFather>

2. 打开后会**提示**"要打开 Telegram Desktop 吗?"此时**点击**"打开Telegram Desktop"如下图所示:

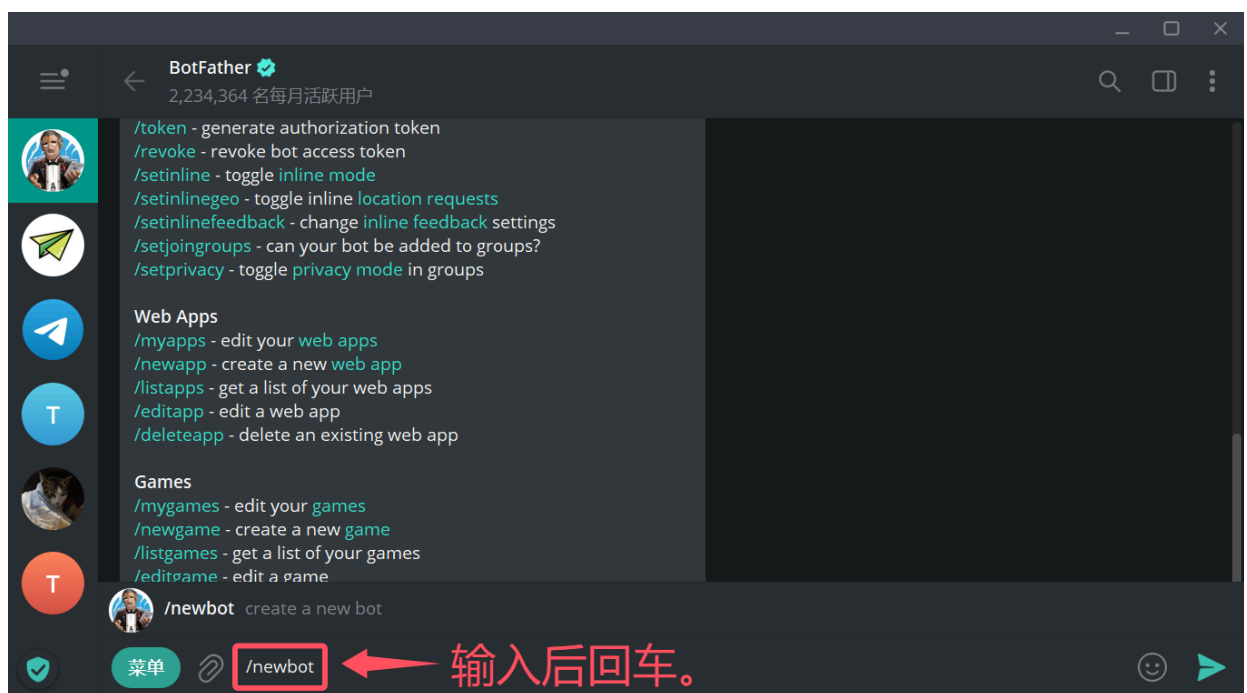


如果没有这个弹窗，说明电脑没有安装Telegram**客户端**，安装后再重试即可。

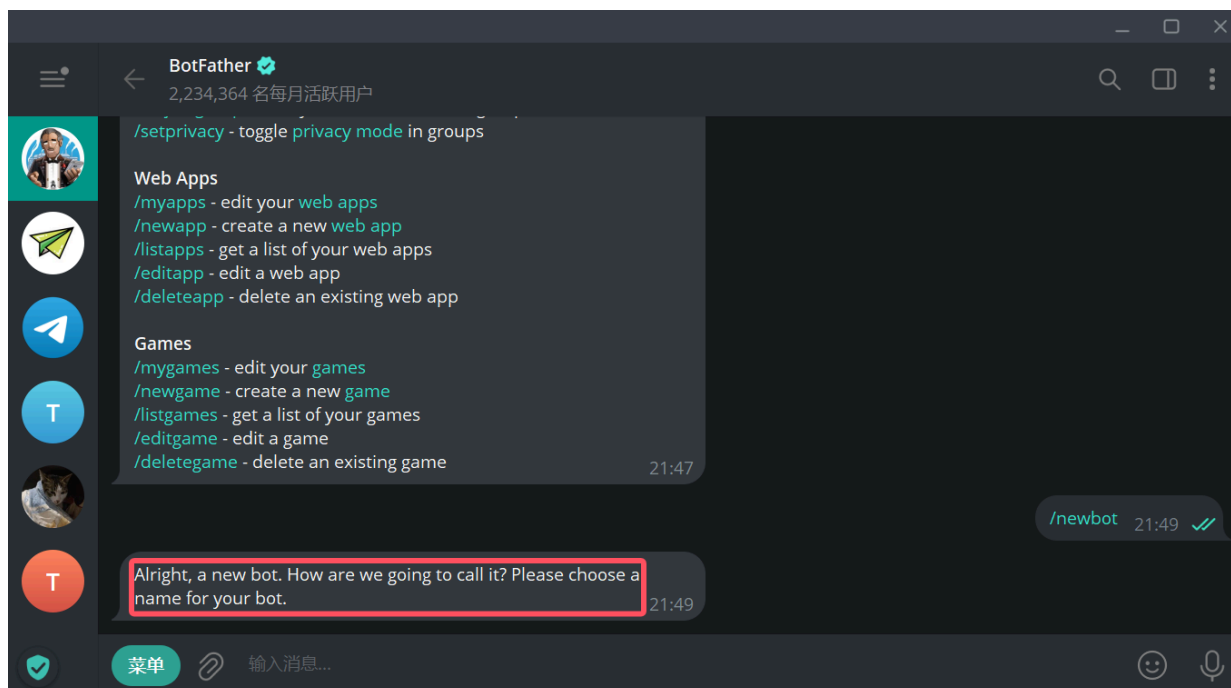
3. **点击开始**，如下图所示：



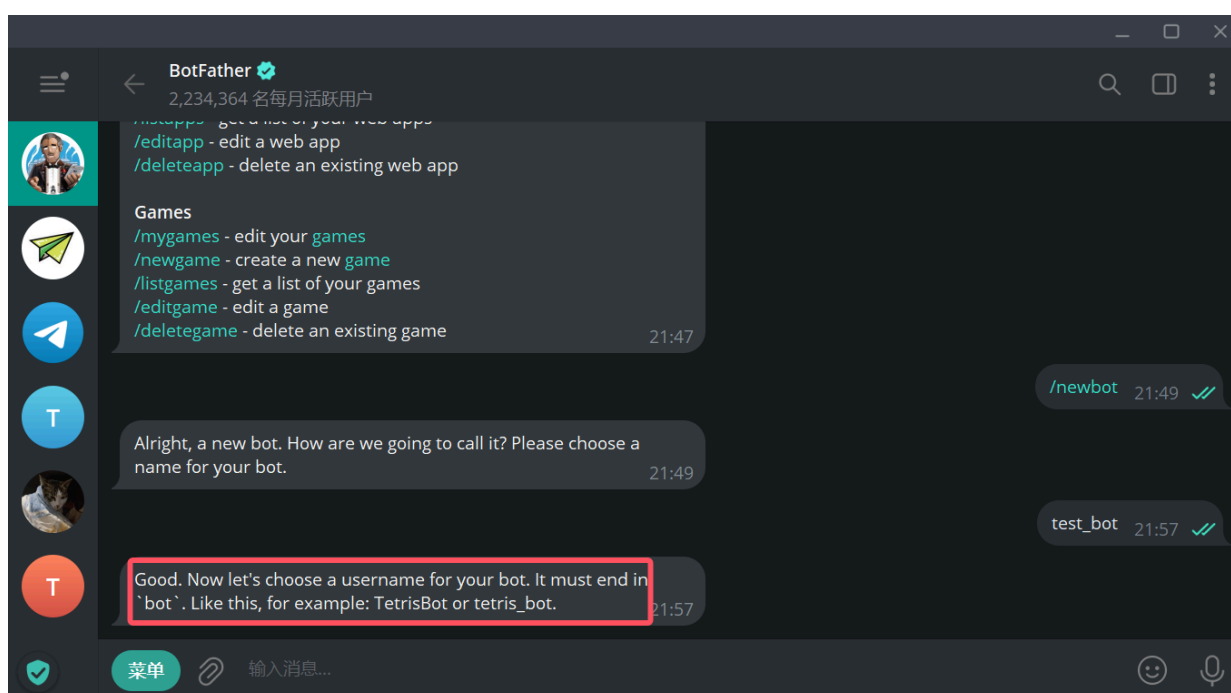
4. 然后在当前聊天框中输入 `/newbot` 后回车，如下图所示：



它会回复你 "Alright, a new bot. How are we going to call it? Please choose a name for your bot." 意思是给机器人取一个名字，如下图所示：

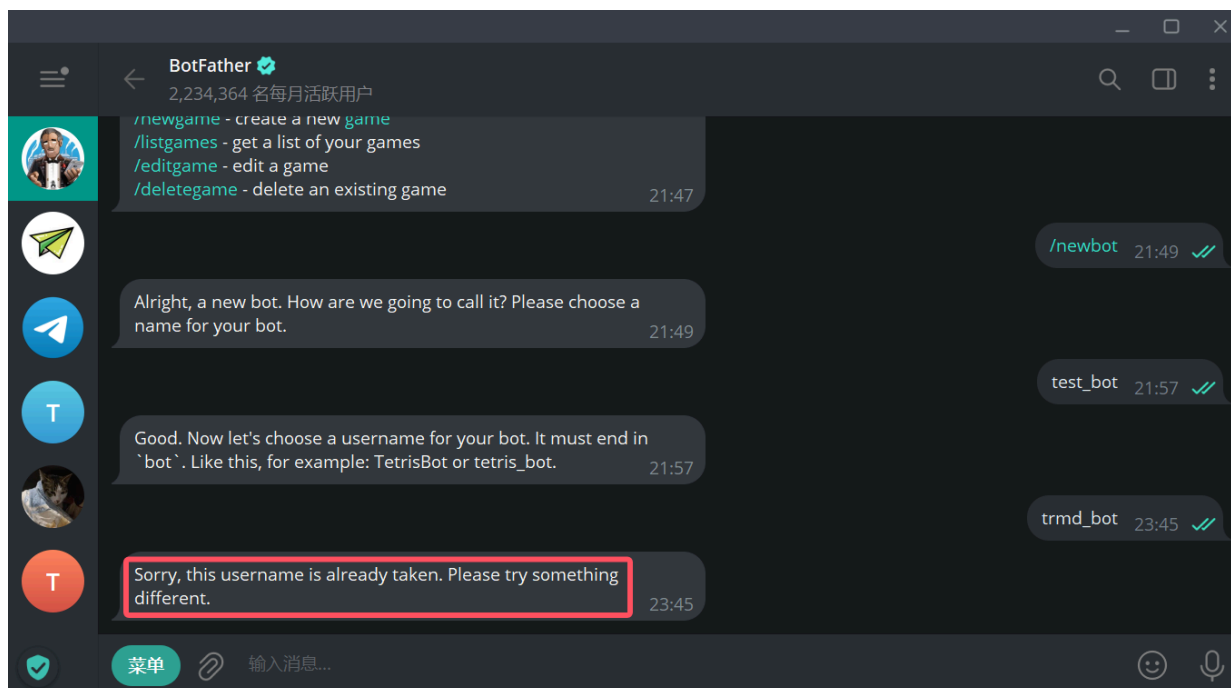


5. 这个名字是显示名称 (display name), 并不是唯一识别码, 随便设置一下即可, 之后可以通过 `/setname` 命令进行修改。



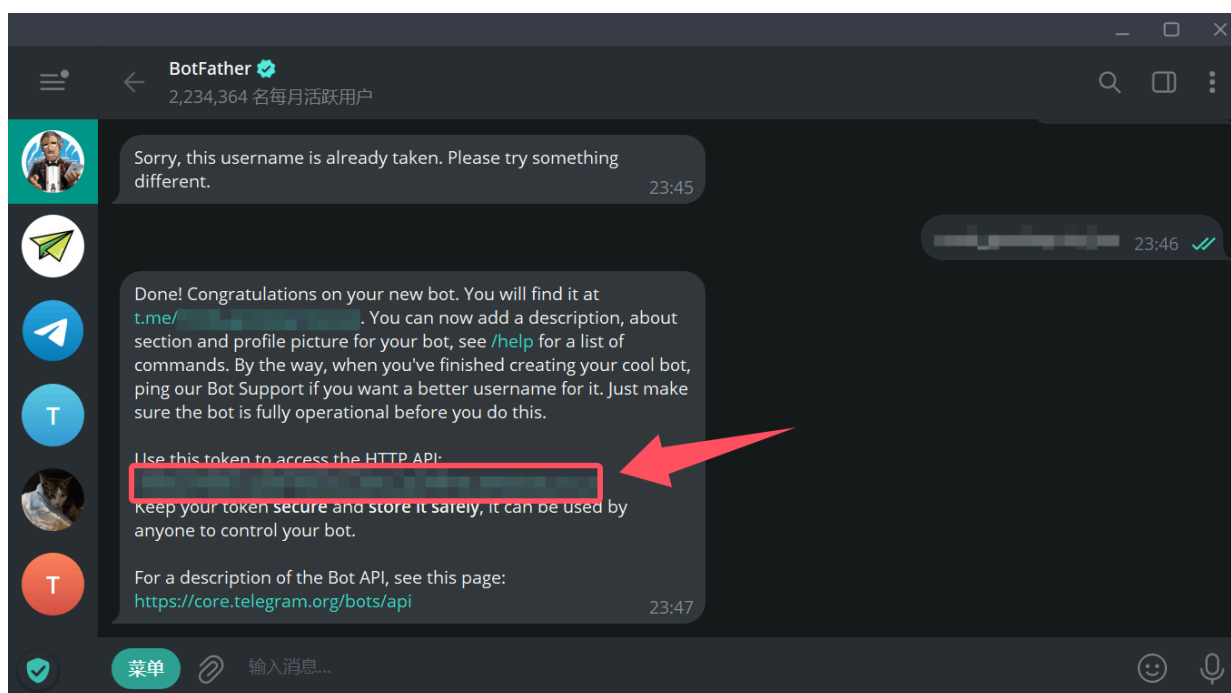
6. 接着设置机器人的**唯一名称**。字符串必须以 `bot` 结尾, 比如 `HelloWorld_bot` 或 `HelloWorldbot` 都是合法的。如果设置的名字已经被占用需要重新设置。如设置成了 `trmd_bot` 但是这个名字已经有人使用了, 此时会提示 "Sorry, this username is already taken. Please try something different." 意思是已经被使用了, 需要拟定一个**不重复的**, 如下图所示:





如果结果如上图所示，则就代表名字重复了，需要重新拟定一个。

7. 直到提示你 "Done! Congratulations on your new bot. . ." 如下图所示：



如果结果如上图所示，则代表 bot\_token 申请成功了，箭头指的红框处就是你所申请的 bot\_token，切记不要泄露给任何人！

## 2.2.2.使用教程:

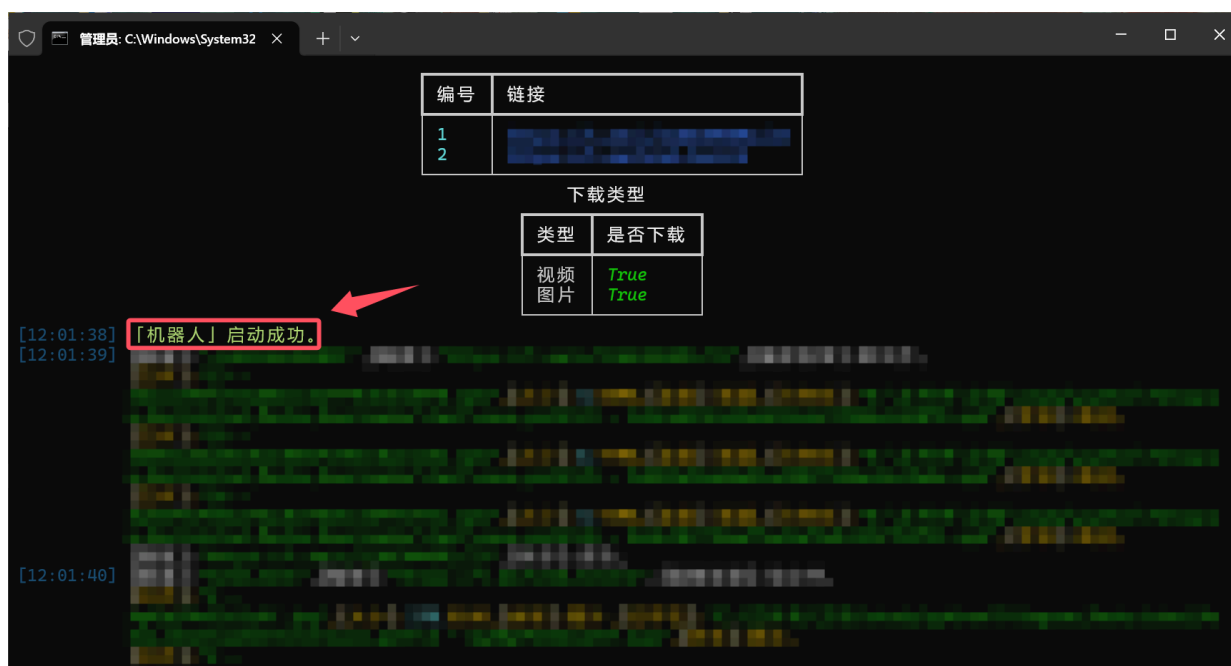
1. 申请完成后，在软件配置时询问"是否启用「机器人」(需要提供bot\_token)? - [y|n] (默认n)"选择 y 代表需要使用，如下图所示：

```
bot_token: 123456:ABC-DEF1234ghIkl-zyx57W2v1u123ew11
download_type: # 需要下载的类型。支持的参数:video,photo。
- video
- photo
is_shutdown: true # 下载完成后是否自动关机。支持的参数:true,false。
links: D:\path\where\your\link\files\save\content.txt # 链接地址写法如下:
# 新建txt文本,一个链接为一行,将路径填入即可请不要加引号,在软件运行前就准备好。
# D:\path\where\your\link\txt\save\content.txt 一个链接一行。
# 不要存在中文或特殊字符。
max_download_task: 3 # 最大的下载任务数,非Telegram会员无效。支持的参数:所有>0的整数。
proxy: # 代理部分,如不使用请全部填null注意冒号后面有空格,否则不生效导致报错。
enable_proxy: true # 是否开启代理。支持的参数:true,false。
hostname: 127.0.0.1 # 代理的ip地址。
is_notice: false # 开关是否询问使用代理的提示。支持的参数:true,false。
scheme: socks5 # 代理的类型。支持的参数:http,socks4,socks5。
port: 10808 # 代理ip的端口。支持的参数:0~65535。
username: null # 代理的账号,没有就填null。
password: null # 代理的密码,没有就填null。
save_path: F:\directory\media\where\you\save # 下载的媒体保存的目录,不要存在中文或特殊字符。

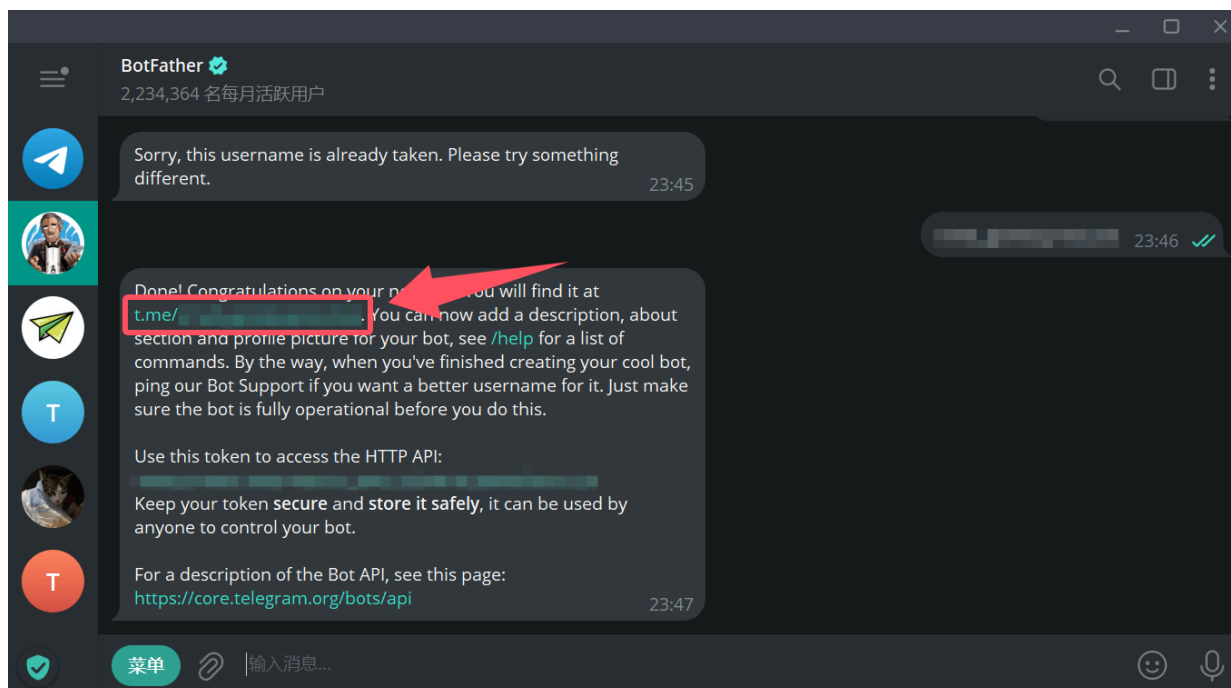
检测到已配置完成的配置文件,是否需要重新配置?(之前的配置文件将为你备份到当前目录下) - 「y|n」(默认n):y
[10:46:49] 原来的配置文件已备份至"
是否需要切换账号? - 「y|n」(默认n):
用户不需要切换「账号」。
「注意」直接回车代表使用上次的记录。
是否启用「机器人」(需要提供bot_token)? - 「y|n」(默认n):y
请配置「bot_token」。
请输入当前账号的「bot_token」上一次的记录是:「[REDACTED]」:|
```

然后在上图箭头所指处填入"2.2.1.申请教程"第7步申请的 bot\_token 后回车,即可配置完成。

2. 在一切配置完成,软件启动成功后等待提示"「机器人」启动成功。",就代表机器人可以使用了,如下图所示:

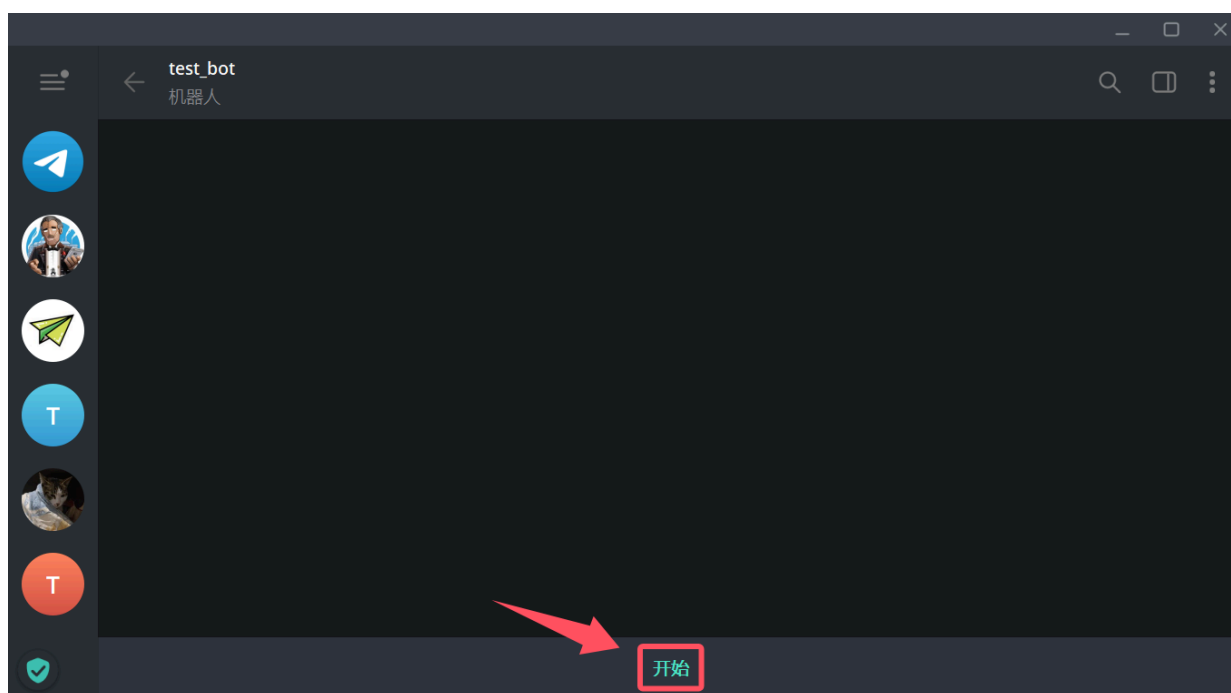


3. 在 Telegram 客户端中找到与 BotFather 的对话框,找到"2.2.1.申请教程"第7步对话的位置(或者用你自己的方式找到你的机器人的对话框),如下图所示:



然后在上图箭头所指处即可跳转到机器人对话框。

4. 点击开始，如下图所示：



不出意外，会收到一条来自机器人发送的消息，如下图所示：



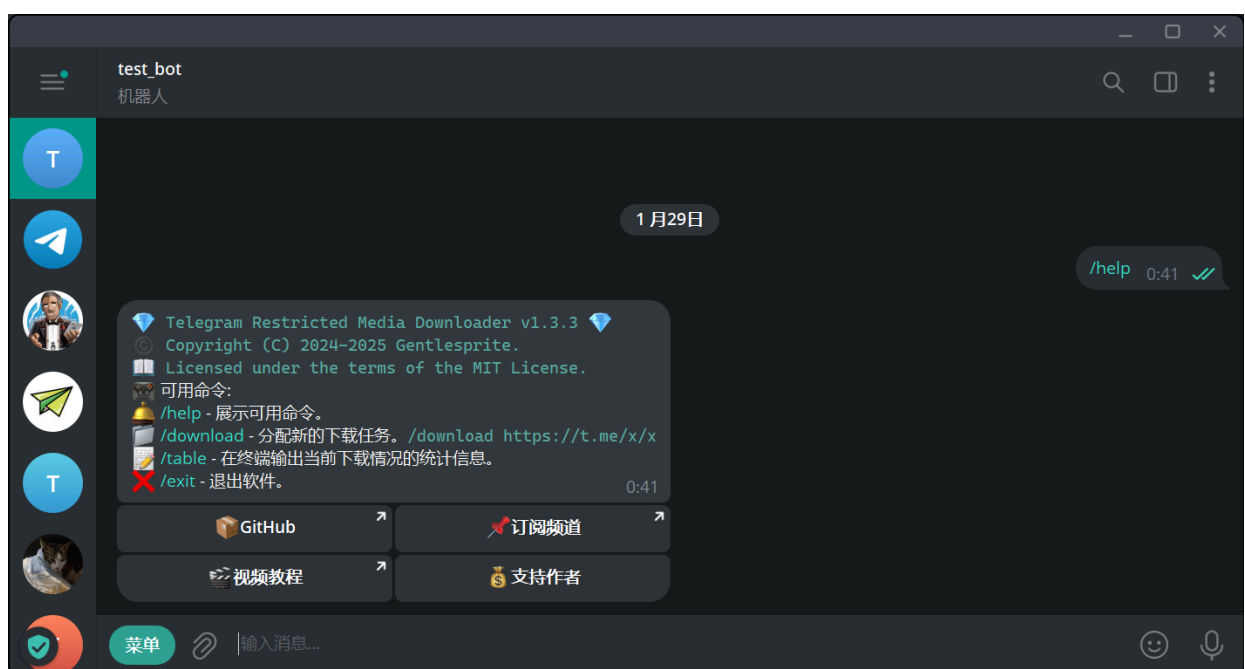
如果没收到尝试给机器人发送任意命令。

5. 目前机器人支持的命令用法及解释如下表所示：

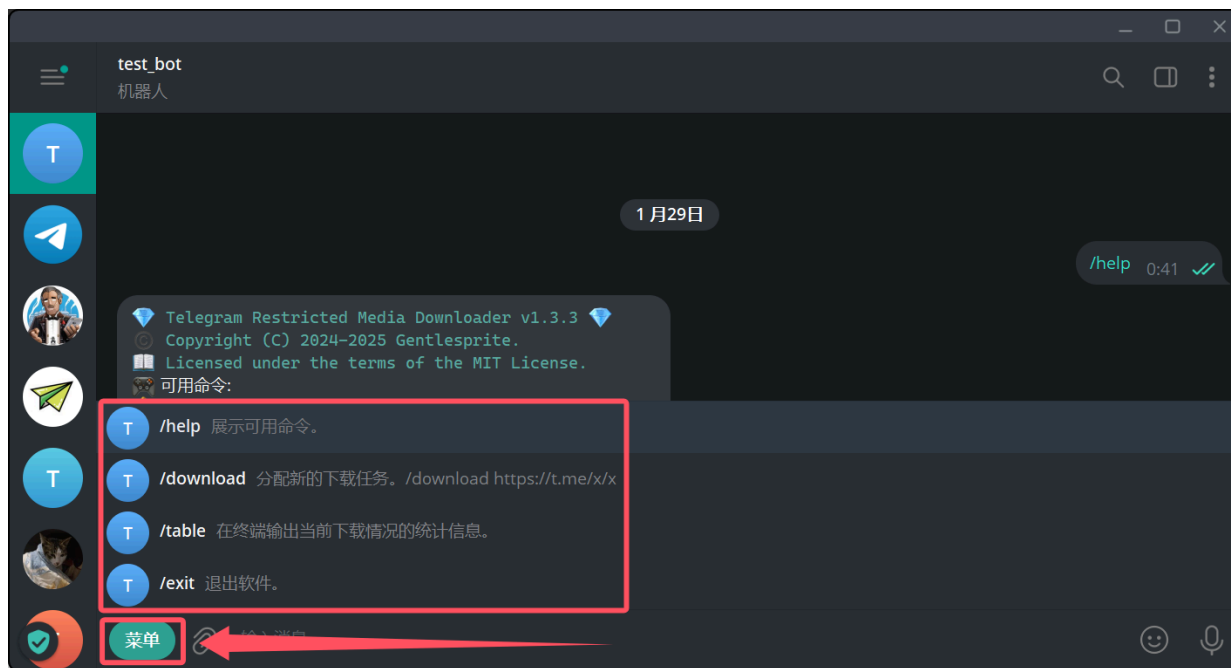
| 命令        | 用法                                               | 解释                                                                         |
|-----------|--------------------------------------------------|----------------------------------------------------------------------------|
| /help     | 向机器人发送发送 /help 即可。                               | 展示 <b>可用命令</b> 。                                                           |
| /download | /download 链接1 链接2 链接3 链接n 或 /download 频道链接 1 100 | 分配 <b>新的</b> 下载任务，两种方式可选( <b>指定链接下载</b> 和 <b>范围下载</b> ，具体使用方法请见下方说明)。      |
| /table    | 向机器人发送 /table 即可。                                | 在 <b>终端</b> 输出 <b>当前</b> 下载情况的 <b>统计信息</b> 。                               |
| /forward  | /forward https://t.me/A https://t.me/B 1 100     | 将 <b>频道A</b> 的消息转发至 <b>频道B</b> ，其中 1 代表 <b>起始ID</b> ，100 代表 <b>截止 ID</b> 。 |

| 命令               | 用法                                                                         | 解释                                         |
|------------------|----------------------------------------------------------------------------|--------------------------------------------|
| /exit            | 向机器人发送 /exit 即可。                                                           | 退出软件。                                      |
| /listen_download | <pre> /listen_download https://t.me/A https://t.me/B https://t.me/n </pre> | 实时监听频道A、频道B和频道n的最新消息(视频和图片)进行下载。           |
| /listen_forward  | <pre> /listen_forward https://t.me/A https://t.me/B </pre>                 | 实时监听频道A的最新消息(任意消息)转发至频道B。但当频道A为私密频道时候无法转发。 |
| /listen_info     | 向机器人发送 /listen_info 即可。                                                    | 查看当前已经创建的监听信息。                             |
| /upload          | /upload 本地文件 目标频道                                                          | 上传本地的文件到指定频道。                              |
| /upload_r        | /upload_r 本地文件夹 目标频道                                                       | 递归上传文件夹(包含子文件夹)到指定频道。                      |
| /download_chat   | /download_chat 频道链接                                                        | 下载指定频道并支持通过内联键盘自定义内容过滤。                    |

6. /help 命令使用教程，如下图所示：



7. 点击**菜单**可以显示机器人可用的命令，如下图所示：



8. /download 命令使用教程，如下图所示：

**⚠ 注意：**

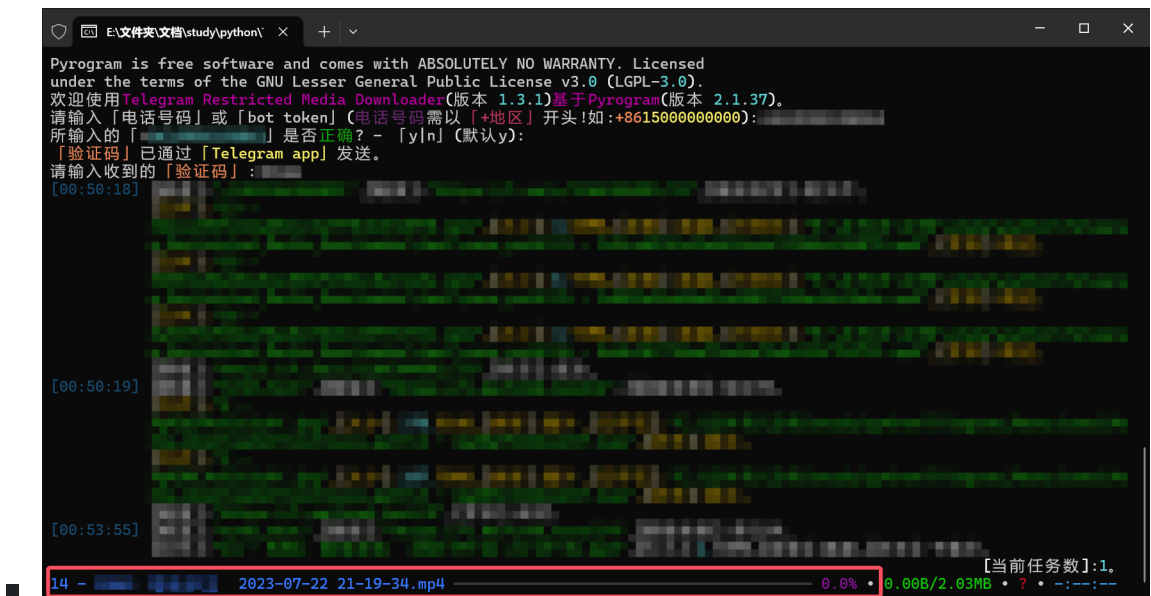
自版本  $\geq v1.6.3$  起：

已全面支持下载时的断点续传功能（支持所有上传场景），增强了在较差网络环境下的传输稳定性与可靠性。

○ 方式一：



■ 只要发送了正确的下载命令，终端就会创建对应的下载任务，如下图所示：



## 方式二：

- 该功能为版本  $\geq 1.5.1$  的新增功能，用于对于**单一链接**的范围下载，并且该方式要求指定**频道链接**、**起始ID**与**结束ID**：

# 语法格式如下：

/download 频道链接 起始ID 结束ID

# 举例：

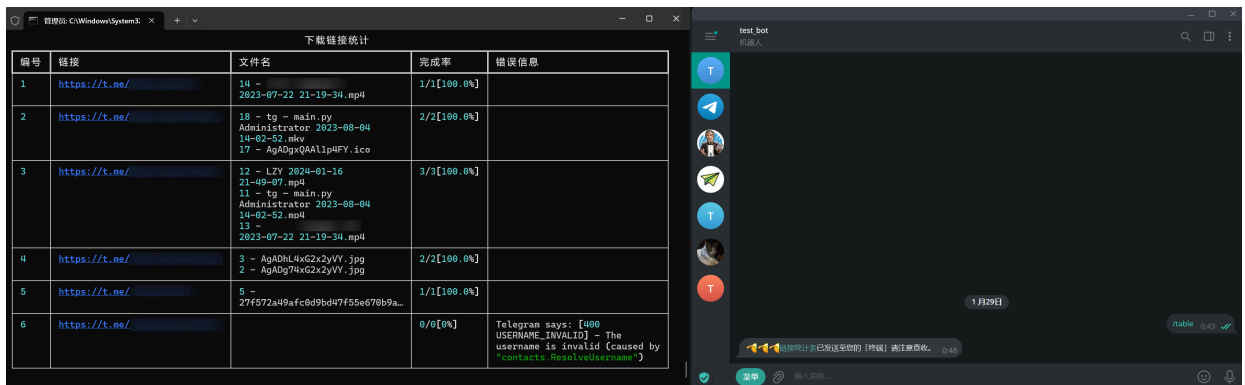
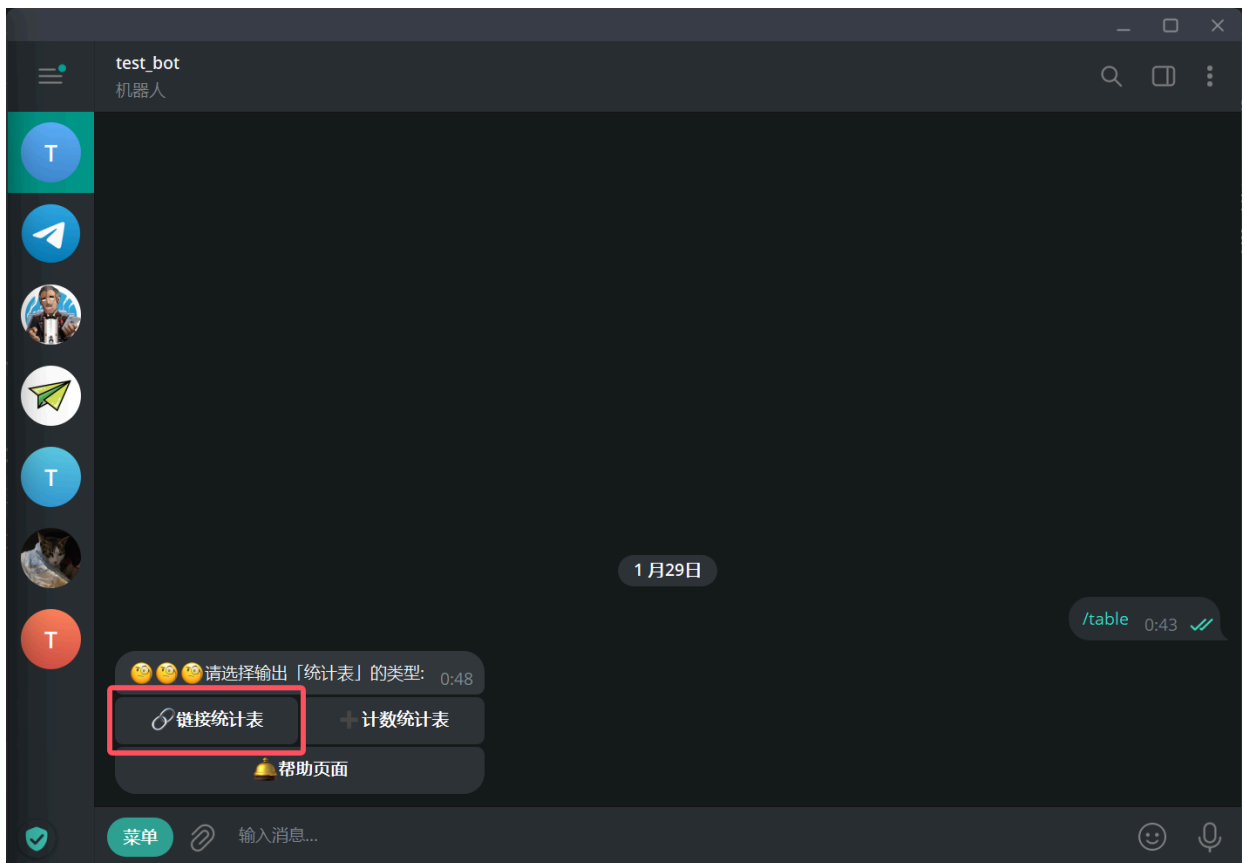
/download https://t.me/test 1 500

# 代表下载https://t.me/test从消息ID=1到结束ID=500的媒体。

## 9. /table 命令使用教程：

需要注意的是，这个表格是**实时的状态**，并不是**最终下载完成的结果**，每一次使用它都会随着**当前的下载记录**而更新。

**链接统计表**的使用，如下图所示：

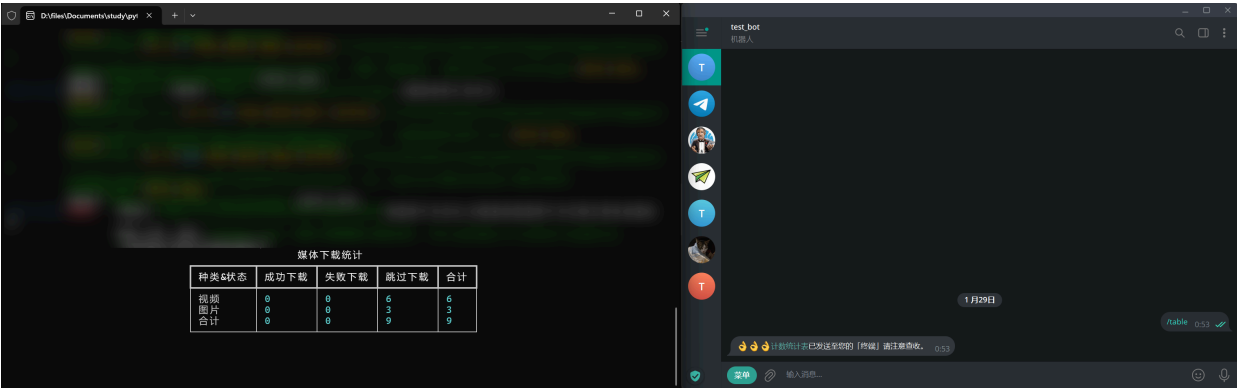
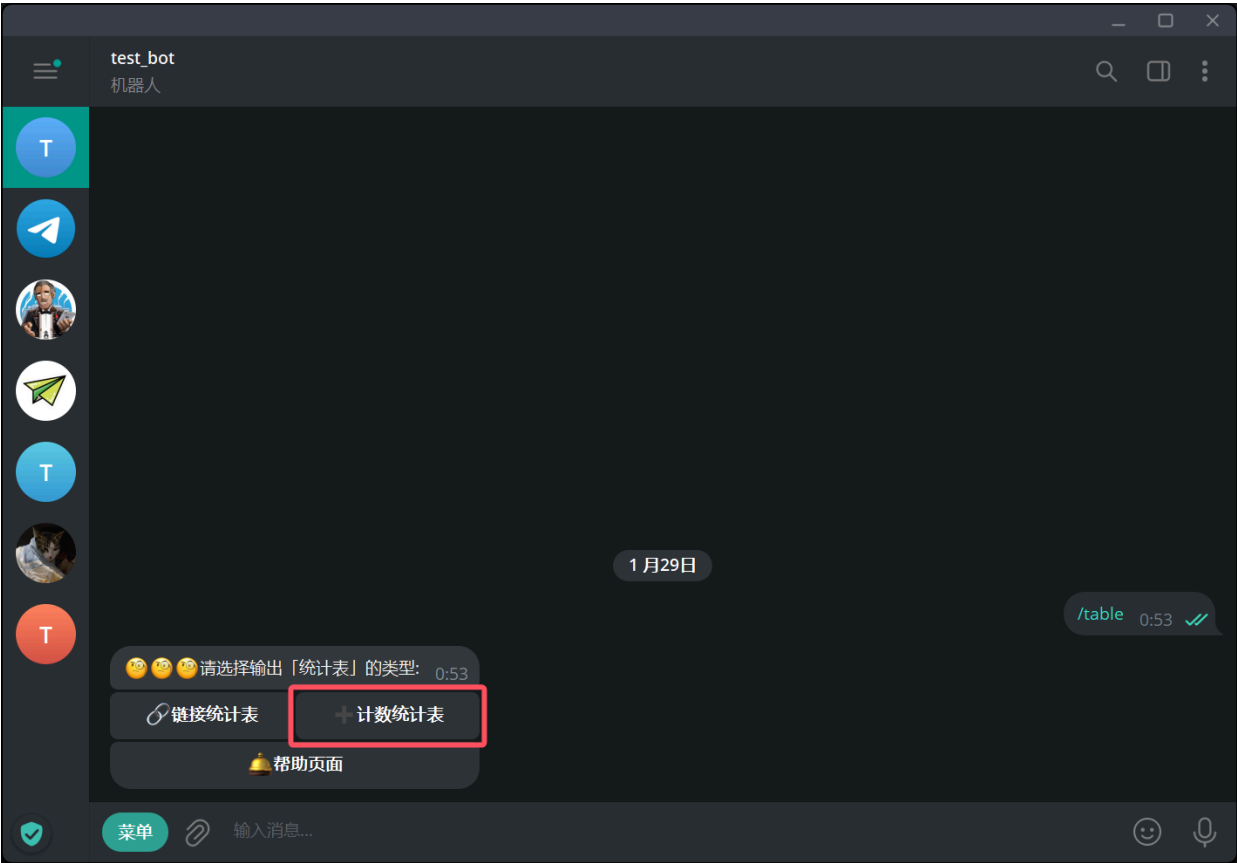


**注意：**由于早期代码设计缺陷，**链接统计表**为后续支持的功能：

- **链接统计表**仅会统计**所有支持的类型**，并不会只统计用户**当前所选择**的类型。
- **链接统计表**对于**评论区媒体**的统计，会出现**总数统计错误**的问题，体现在**总数为1，小于当前的下载数，完成率 >>100%**的问题(该问题已在  $\geq v1.5.9$  修复)。
- 当用户**未选择下载所有支持的类型**时，在**用户所选择的类型**下载完成后(或使用机器人发送**链接统计表**)，尽管所有用户指定类型的文件已经下载完成，当**链接统计表**显示**完成率不为100%**时，代表该链接还存在其他用户未指定的文件类型，但实际用户所指定的类型已经下载完成了，是正常情况。
- 版本  $\geq v1.6.5$  起已支持**导出表格**功能，通过该命令可在运行时控制**是否在退出后导出指定类型的表格**。



计数统计表的使用，如下图所示:



10. /forward 命令使用教程:

**⚠ 注意：**

消息能否转发，在于频道是否开启了 `限制保存内容` 功能。

如果**无法转发**，机器人会在聊天框提供一个**下载按钮与下载后上传按钮** ( $\geq v1.6.7$ )。

自版本  $\geq v1.7.5$  起：

为确保"受限转发"功能顺利完成，在**下载后上传过程中**，将**忽略配置文件中设置的下载类型限制**，仅遵循 [上传设置] 中的类型过滤规则。

自版本  $\geq v1.6.9$  起：

`/forward` 将支持过滤转发类型。

可通过 [帮助页面] -> [设置] -> [转发设置] 进行修改。

- 转发消息语法：

```
/forward 频道A 频道B 起始ID 结束ID
```

- 实例：

- 该实例代表转发 `https://t.me/test` **频道**中从 消息ID=1 到 结束ID=500 的消息到 `https://t.me/test2` **频道**。

```
/forward https://t.me/test https://t.me/test2 1 500
```

- 转发至个人收藏夹（任选一种命令即可）：

- 方式1，使用me或self指向个人收藏夹（任选一种命令即可）：

```
/forward https://t.me/test me 1 500
```

```
/forward https://t.me/test self 1 500
```

- 方式2，使用 `https://t.me/用户名`：

- 用户名即指个人账户信息里@后面那一串，需用户提前手动自己设置，否则无法使用该方式。
- 例如通过查询个人信息，得知当前设置的用户名为 `@developer`，此时指向个人的链接即为 `https://t.me/developer`。

```
/forward https://t.me/test https://t.me/developer 1 500
```

- 无论使用方式1或方式2，都代表转发 `https://t.me/test` 频道 中从 消息ID=1 到结束 ID=500 的消息到个人收藏夹。
- 转发个人收藏夹的内容至任意频道：
  - 方式1：
 

```
/forward me https://t.me/test 1 500
```
  - 方式2：
 

```
/forward https://t.me/developer me 1 500
```
  - 无论使用方式1或方式2，都代表转发个人收藏夹中从 消息ID=1 到结束 ID=500 的消息到 `https://t.me/test` 频道。

11. `/exit` 命令使用教程，如下图所示：



12. `/listen_download` 命令使用教程：

- `/listen_download` 监听下载用于，实时监听该链接的最新消息进行下载。
  - 在用户发送了正确的监听命令后，会收到机器人的成功提示。
  - 当被监听的频道有可下载的内容时，就会自动发送命令下载。
- 注册监听下载：**

```
/listen_download https://t.me/A
```

- **注销监听下载：**

**再次向机器人发送创建监听时的命令，机器人将会提供给用户一个用于注销监听的内联键盘，点击确认即可。**

```
/listen_download https://t.me/A
```

- **注册多个监听下载：**

```
/listen_download https://t.me/A https://t.me/B https://t.me/n
```

- **注销多个监听下载：**

```
/listen_download https://t.me/A https://t.me/B https://t.me/n
```

### 13. /listen\_forward 命令使用教程：

 **注意：**

自版本  $\geq v1.6.7$  起：

当检测到"受限转发"时，自动采用"下载后上传"的方式(默认**开启**)。

当**下载并完成上传后**，可选择**是否删除本地文件**(默认**关闭**)。

并且可通过 [帮助页面] -> [设置] -> [上传设置] 进行修改。

自版本  $\geq v1.7.5$  起：

为确保"受限转发"功能顺利完成，在**下载后上传过程中**，将**忽略配置文件中设置的下载类型限制**，仅遵循 [上传设置] 中的类型过滤规则。

自版本  $\geq v1.6.9$  起：

/listen\_forward 将支持过滤转发类型。

可通过 [帮助页面] -> [设置] -> [转发设置] 进行修改。

- /listen\_forward 监听转发用于，实时监听该链接的最新消息。
  - 与 /forward 命令一样，消息能否转发，在于频道是否开启了 限制保存内容 功能。
  - 但在  $\geq v1.6.7$  起可以通过下载再上传的方式进行"转发"。
  - 在用户发送了正确的监听命令后，会收到机器人的成功提示。
  - 当被监听的频道有**任何**新内容时，就会自动转发至用户所指定的频道。

## • 监听行为说明：

### ○ 频道独占原则：

- 每个频道**同一时间**只能激活一种监听模式（下载或转发）。
- 对于 `/listen_forward` 命令，"同一频道"特指**被监听**的源频道。
- 转发目标频道**不受此限制**，仍可通过 `/listen_download` 创建下载任务。

### ○ 操作限制：

- 当 频道A 正在监听转发 频道B 时：
  - 可以**同时**在 频道B 设置监听下载。
  - 但 频道B 的监听下载，不会响应来自 频道A 的监听转发。

### ○ 监听切换流程：

- 必须先通过**同一命令**(注册监听时的命令)来取消现有监听。
- 然后才能创建**新的监听**事件。

## • 注册监听转发：

```
/listen_forward https://t.me/A https://t.me/B
```

## • 注销监听转发：

**再次向机器人发送创建监听时的命令，机器人将会提供给用户一个用于注销监听的内联键盘，点击确认即可。**

```
/listen_forward https://t.me/A https://t.me/B
```

## 14. `/listen_info` 命令使用教程：

- `/listen_info` 用于查看**当前已经创建的监听信息**，**直接发送即可**：

```
/listen_info
```

## 15. `/upload` 命令使用教程：

### ⚠ 注意:

自版本  $\geq v1.7.9$  起:

已全面支持上传时的断点续传功能 (支持所有上传场景), 增强了在较差网络环境下的传输稳定性与可靠性。

自版本  $\geq v1.8.4$  起:

`/upload` 命令支持 `me`、`self` 作为目标频道的参数 (`me`、`self` 作为目标频道参时, 代表指向个人收藏夹 Saved Messages )

- `/upload` 用于上传本地的文件到指定频道。
  - 支持上传文件夹 (当指定路径为文件夹时并且版本需 $\geq v1.7.1$ )。
  - 当指定的路径为**文件夹**时, 将会上传指定文件夹下**所有**的文件。
- 上传文件语法:

`/upload` 本地文件(夹) 目标频道

### 注意:

Telegram 对上传的**单个文件**大小设有**明确限制**:

- **普通用户**: **单个文件**最大上传大小为 2000 MiB (约 2 GB)
- **会员用户** (Telegram Premium) : **单个文件**最大上传大小为 4000 MiB (约 4 GB)

### 对于Windows用户:

#### 上传一个文件:

```
/upload C:\files\video.mp4 https://t.me/test
```

#### 上传一个文件夹 ( $\geq v1.7.1$ ) :

```
/upload C:\files https://t.me/test
```

### 对于Linux用户:

#### 上传一个文件:

```
/upload /home/username/files/video.mp4 https://t.me/test
```



## 上传一个文件夹 (≥v1.7.1) :

```
/upload /home/username/files https://t.me/test
```

### 16. /upload\_r 命令使用教程:

#### 注意:

自版本 ≥v1.8.4 起:

/upload\_r 命令支持 me 、 self 作为目标频道的参数 ( me 、 self 作为目标频道参时, 代表指向个人收藏夹 Saved Messages ) 。

- /upload\_r 命令是 /upload 命令的扩展版本, 专为**批量文件上传**场景设计, 支持递归处理目录结构, 与 /upload 不同的是:

- /upload\_r 命令接收**文件夹路径**作为其第二个参数, 自动**遍历该目录及其所有子目录**中的文件。

```
/uploadr C:\files https://t.me/test
```

- 当**第二个参数为文件路径**时, 自动切换为 /upload 的单文件上传模式。

```
/uploadr C:\files\video.mp4 https://t.me/test
```

```
/upload C:\files\video.mp4 https://t.me/test
```

*以上两个命令在功能上完全等效, 系统会自动识别文件类型并采用相应的上传策略。*

### 17. /download\_chat 命令使用教程:

- /download\_chat 下载指定频道。
  - 与 /download 不同的是: /download\_chat 支持通过机器人发送的内联键盘进行自定义内容过滤。
  - 该命令支持下载无法复制具体请求链接的对话。
  - 目前该功能支持按日期范围、文件类型来过滤要下载的内容。
  - 使用该命令后, **需要通过操作机器人回复中的内联键盘**, 来设置过滤器、执行任务或取消任务。

- 需要注意的是，在上一个 `/download_chat` 命令任务未执行或取消前，无法发起新的 `/download_chat` 命令来创建下载任务。
- 自版本  $\geq v1.7.5$  起，`/download_chat` 命令创建的下载任务过滤条件将完全遵循用户在内联键盘中的设置，意味着该命令不会遵循配置文件中的任何规则（例如配置文件中的下载文件类型设置）。

- 下载指定频道语法：

`/download_chat` 频道链接

- 发送命令后，可**根据个人需求**设置过滤器（可选）。
- 在设置完成后，必须**手动点击**"执行任务"或"取消任务"以继续或终止流程，否则该命令将**始终处于等待状态，并阻塞新的 `/download_chat` 命令**。

- 下载用户与机器人的对话媒体：

用户与机器人的对话无法复制具体的请求链接，因此该方法用于解决某些用户与机器人对话中禁止用户进行转发、下载的情况。

- 首先要获取机器人用户名称，在聊天对话框中点击机器人名称查看其用户名，此处假设机器人名称为 `@developer_bot`。
- 拼接机器人的请求链接为 `https://t.me/developer_bot`。

`/download_chat https://t.me/developer_bot`

- 下载个人收藏夹（任选一种命令即可）：

- 方式1（任选一种命令即可）：

`/download_chat me`

`/download_chat self`

- 方式2：

- 首先要获取用户名称，此处假设用户名已设定，个人信息中查看用户名为 `@developer`。
- 拼接个人收藏夹的请求链接为 `https://t.me/developer`。



/download\_chat https://t.me/developer

## 2.3.配置文件说明:

### 用户配置文件:

#### ① Note

用户配置文件通常无需自行配置, 此处旨在介绍全局配置文件中各参数的含义。用户配置文件会在软件启动时自动在软件目录下生成 config.yaml 文件, 并详细地引导用户配置参数。

**手动编辑** config.yaml 对配置进行修改时, 请仔细阅读下面内容理解各参数含义。需注意配置文件使用的引号、冒号均为半角, 冒号后需有一个空格。

```
api_hash: xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx # 申请的api_hash。
api_id: 'xxxxxxx' # 申请的api_id。
# bot_token(选填)如果不填,就不能使用机器人功能。可前往https://t.me/BotFather免费申请。
bot_token: 123456:ABC-DEF1234ghIkl-zyx57W2v1u123ew11
download_type: # 需要下载的类型。支持的参数:video,photo,document,audio,voice,animation。
- video # 视频。
- photo # 图片。
- document # 文档。
- audio # 音频。
- voice # 语音。
- animation # GIF。
is_shutdown: true # 下载完成后是否自动关机。支持的参数:true,false。
links: D:\path\where\your\link\files\save\content.txt # 链接地址写法如下:
# 新建txt文本,一个链接为一行,将路径填入即可请不要加引号,在软件运行前就准备好。
# D:\path\where\your\link\txt\save\content.txt 一个链接一行。
max_retries:
  download: 5 # 最大的下载任务的重试次数。
  upload: 3 # 最大的上传任务的重试次数。
max_tasks:
  download: 5 # 最大同时下载的任务数。
  upload: 3 # 最大同时上传的任务数。
proxy: # 代理部分,如不使用请全部填null注意冒号后面有空格,否则不生效导致报错。
  enable_proxy: true # 是否开启代理。支持的参数:true,false。
  hostname: 127.0.0.1 # 代理的ip地址。
  scheme: socks5 # 代理的类型。支持的参数:http,socks4,socks5。
  port: 10808 # 代理ip的端口。支持的参数:0~65535。
  username: null # 代理的账号,没有就填null。
  password: null # 代理的密码,没有就填null。
```

`save_directory`: `F:\directory\media\where\you\save` # 下载的媒体保存的目录(支持通配符, 不支持网络路径)。

## 自版本 $\geq v1.7.4$ 起, `save_directory` 将支持通配符。

通配符允许用户动态生成存储路径。系统会在下载时自动将通配符替换为对应的实际值, 实现按规则自动分类存储。

- 目前 `save_directory` 支持的通配符如下表所示:

| 通配符                      | 意义                 |
|--------------------------|--------------------|
| <code>%CHAT_ID%</code>   | 以实际 频道ID 作为指定路径填充。 |
| <code>%MIME_TYPE%</code> | 以实际 文件类型 作为指定路径填充。 |

- 用法示例1:

```
F:\directory\media\%CHAT_ID%
```

- 用法示例2:

```
F:\directory\media\%MIME_TYPE%
```

- 用法示例3:

```
F:\directory\media\%CHAT_ID%\%MIME_TYPE%
```

## 全局配置文件:

### Note

全局配置文件通常无需自行配置, 此处旨在介绍全局配置文件中各参数的含义。部分参数支持通过机器人设置进行修改。

`console_log_level`: `WARNING` # 在终端显示的最低日志类型。

`export_table`:

`count`: `false` # 控制运行结束时是否导出下载计数统计表。

`link`: `false` # 控制运行结束时是否导出下载链接统计表。

`file_log_level`: `INFO` # 在 `TRMD_LOG.log` 文件中记录的最低日志类型。

`forward_type`: # 控制 `/listen_forward` 与 `/forward` 命令可转发的文件类型。

`animation: true` # GIF类型。  
`audio: true` # 音频类型。  
`document: true` # 文档类型。  
`photo: true` # 图片类型。  
`text: true` # 文本消息类型。  
`video: true` # 视频类型。  
`voice: true` # 语音类型。  
`notice: false` # 控制机器人启动时候是否发送启动通知。  
`upload:`  
    `delete: false` # 控制/`listen_forward`命令遇到受限内容时,下载上传完成后是否删除已上传完成的本地文件。  
    `download_upload: true` # 控制/`listen_forward`命令遇到受限内容时,是否下载后上传到指定的转发频道。

## 全局配置文件存放路径:

### 对于Windows用户:

`%APPDATA%/TRMD/.CONFIG.yaml`

### 对于Linux用户:

`~/.config/TRMD/.CONFIG.yaml`

## 2.4.使用注意事项:

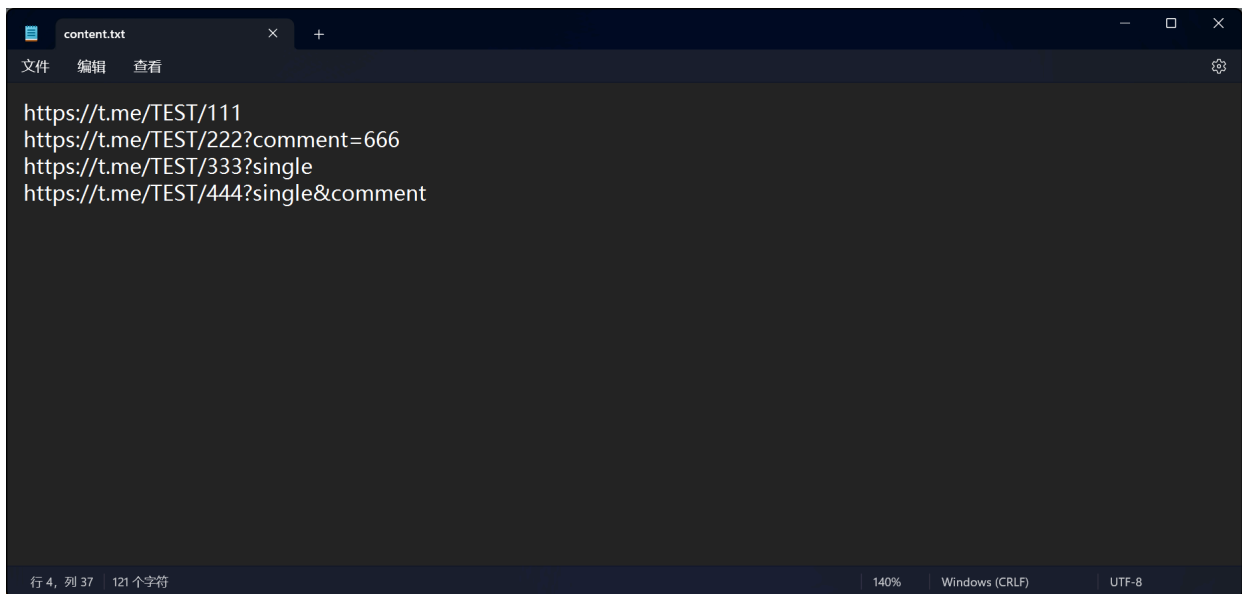
---

### ▼ 点击展开

1. 链接获取方法: 对想要保存的媒体文件点击**鼠标右键**然后选择**复制消息直链**如下图所示:

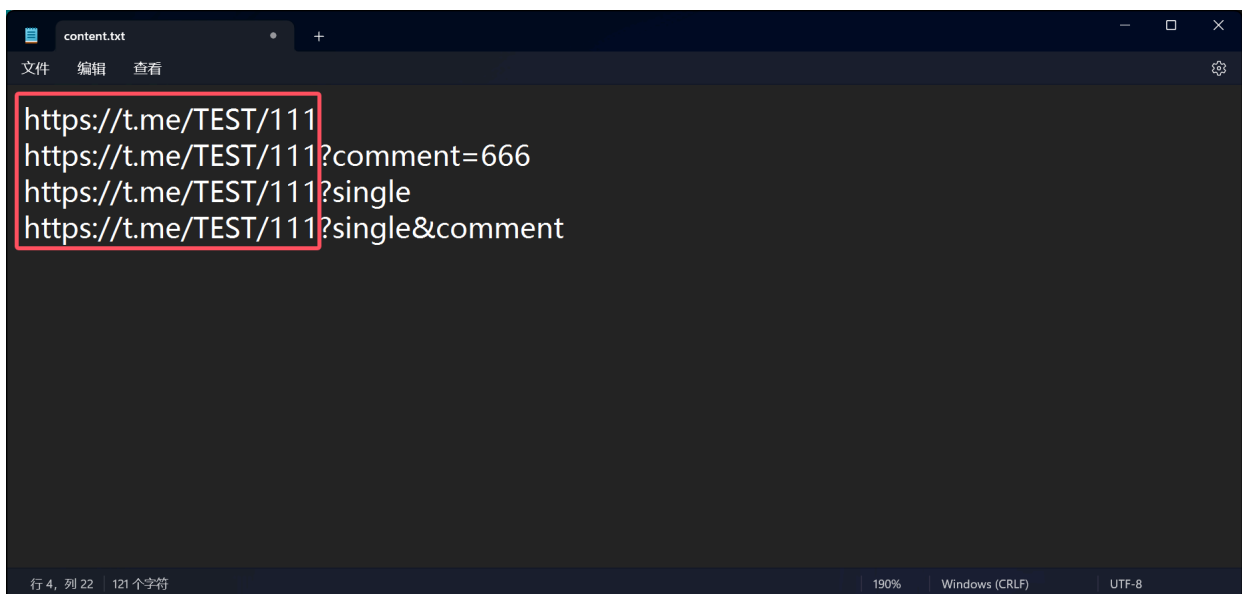


2. 目前支持**视频**和**图片**两种类型的下载。
3. 如果当前复制的**链接**为多张图片或视频，那么程序会**自动下载当前消息所有的内容**!
4. 要下载评论区里的视频或图片，请直接打开评论区，找到任意一个视频或图片，复制它的消息直链(链接末尾会带 `?comment=123456` 这样的参数，不要删除它)。这个链接可以用来下载评论区里的所有视频和图片。注意最好不要手动在链接后面添加 `?comment=` 参数，推荐通过复制的方式获取正确链接，否则可能会错误地解析成正文内容。
5. links的文本**写法**1如下图所示：



6. 你所需要下载的视频前提是你当前的Telegram账号，在此视频链接的频道中，否则会报错无法下载！！！！

7. 常见的错误写法：



## 重复链接问题说明:

### 问题描述:

- 如上图所示，在提交的下载任务中，存在多个前缀相同但参数不同的链接（如 ?comment 、 ?single 或 ?single&comment ）。

### 问题原因:

- 当链接包含 ?comment 参数时，会自动下载原始消息及其评论区内容。

- 如果同时提交**相同前缀但无 ?comment 的链接**，会导致**同一资源被重复添加**至下载队列。
- 若前一次任务尚未完成，重复提交相同资源会触发**任务冲突**，进而引发下载异常。

**解决方案:**

- **仅需提交一个完整链接**（如带 ?comment 的版本），系统会自动处理**原始内容及评论区**，无需额外提交无参数版本。
- **避免重复提交相同资源**，确保每条链接的 t.me/c/<频道>/<消息ID> 部分唯一，防止任务冗余。
- 由于这些链接的**频道名和消息ID**完全一致，实际上指向的是**同一资源**的不同表现形式。
- 但请注意以下参数的功能差异：
  - ?comment 用于标识是否下载**评论区**内容。
  - ?single 用于在**媒体组**中指定仅下载该消息ID对应的单个资源。
  - ?single&comment 用于指定仅下载评论区中该消息ID所对应的内容。

Telegram 字段解释如下表所示：

| 字段                                | 解释                                        |
|-----------------------------------|-------------------------------------------|
| ?comment                          | <b>评论区</b> 的链接。                           |
| ?single                           | <b>单独</b> 的链接。                            |
| ?single&comment                   | <b>评论区中单独</b> 的链接。                        |
| /c                                | <b>私密频道</b> 的链接。                          |
| https://t.me/TEST/111/666         | 频道 TEST <b>话题</b> 111 的链接。                |
| https://t.me/c/1111111111/333/666 | <b>私密频道</b> 1111111111 <b>话题</b> 333 的链接。 |

**链接解释说明:**

Telegram 链接组成如下表所示:

| 频道类型 | 链接组成                            |
|------|---------------------------------|
| 正常频道 | https://t.me/频道名/消息ID           |
| 私密频道 | https://t.me/c/频道名(10位纯数字)/消息ID |
| 话题频道 | https://t.me/频道名/话题ID/消息ID      |

| 频道类型   | 链接组成                                 |
|--------|--------------------------------------|
| 私密话题频道 | https://t.me/c/频道名(10位纯数字)/话题ID/消息ID |

Telegram 链接所有链接格式如下表所示:

"所有"指的是如果有**合并发送为一组**的文件，则给定一个链接，所有**合并发送的文件**会被全部下载。

"媒体"指的是**视频**和**图片**。

| 链接                                       | 实际频道名          | 消息ID | 解释                                                            |
|------------------------------------------|----------------|------|---------------------------------------------------------------|
| https://t.me/TEST/111                    | TEST           | 111  | 下载该链接的 <b>所有媒体</b> 。                                          |
| https://t.me/TEST/111?single             | TEST           | 111  | 下载该链接的 <b>对应的一个媒体</b> 。                                       |
| https://t.me/TEST/111?comment=666        | TEST           | 111  | 下载该链接的 <b>视频图片</b> 的同时，下载该链接下方的 <b>评论区</b> 的 <b>对应的一个媒体</b> 。 |
| https://t.me/TEST/111?single&comment=666 | TEST           | 111  | 下载该链接下方的 <b>评论区</b> 的 <b>对应的一个媒体</b> 。                        |
| https://t.me/c/1111111111/666            | -1001111111111 | 666  | 下载该 <b>私密频道</b> 链接的 <b>所有媒体</b> 。                             |
| https://t.me/TEST/111/666                | TEST           | 666  | 下载该 <b>话题</b> 链接的 <b>所有媒体</b> 。                               |

| 链接                                                                                | 实际频道名           | 消息 ID | 解释                       |
|-----------------------------------------------------------------------------------|-----------------|-------|--------------------------|
| <a href="https://t.me/c/1111111111/333/666">https://t.me/c/1111111111/333/666</a> | -10011111111111 | 666   | 下载该 <b>私密话题</b> 链接的所有媒体。 |

评论区链接的下载行为规则的说明:

- **标准链接 (无 ?comment 参数) :**
  - 仅下载**消息正文内容** (即频道/群组中直接发布的原始消息) 。
  - **不包含评论区内容**，即使原消息存在评论，也不会被纳入下载任务。
- **带 ?comment 参数的链接:**
  - 下载**消息正文 + 关联的全部评论区内容** (完整会话结构) 。
  - 若原消息**无评论区** (如频道消息或评论功能关闭) ，则**仅下载消息正文内容**，与无参数版本行为一致。
- 非下载评论区的**推荐**写法如下表所示:

| 频道类型   | 链接                                                                                |
|--------|-----------------------------------------------------------------------------------|
| 正常频道   | <a href="https://t.me/xxx/111">https://t.me/xxx/111</a>                           |
| 私密频道   | <a href="https://t.me/c/xxxxxxxxxx/111">https://t.me/c/xxxxxxxxxx/111</a>         |
| 话题频道   | <a href="https://t.me/xxx/xxx/111">https://t.me/xxx/xxx/111</a>                   |
| 私密话题频道 | <a href="https://t.me/c/xxxxxxxxxx/xxx/111">https://t.me/c/xxxxxxxxxx/xxx/111</a> |

关于 ?single 及 ?single&comment 参数的下载行为说明 (v1.5.8+) :

功能变更概述:

自 ≥v1.5.8 版本起，链接中包含 ?single 或 ?single&comment 参数时，系统将启用**单文件下载模式**。此模式专为以下场景设计与优化：

- 解决 ≥1.5.8 版本 /listen\_download 当监听到合并发送的文件时，出现**重复下载问题**。
- 用户需求，仅需从合并发送的多媒体组中提取**特定单一文件**。



- 用户需求，仅需下载评论区中的**单个指定媒体**（避免评论区媒体过多时，迟迟下载不到想要的文件）。

参数行为详解:

| 参数格式                   | 下载范围                       | 应用场景            |
|------------------------|----------------------------|-----------------|
| xx?single              | 仅下载消息正文中的xx <b>对应媒体文件</b>  | 从合并图组/视频组提取单文件  |
| ?<br>single&comment=xx | 仅下载评论区中的xx <b>所对应的媒体文件</b> | 获取评论区单独分享的图片/视频 |

版本兼容性说明:

- 此特性**仅对 v1.5.8 及以上版本**生效。
- 历史版本中这些参数可能被忽略，导致完整内容下载。

最佳实践建议:

- **单一文件提取:** 当消息包含多个媒体文件时，使用标准链接附加 ?single 参数可精准获取首个文件：

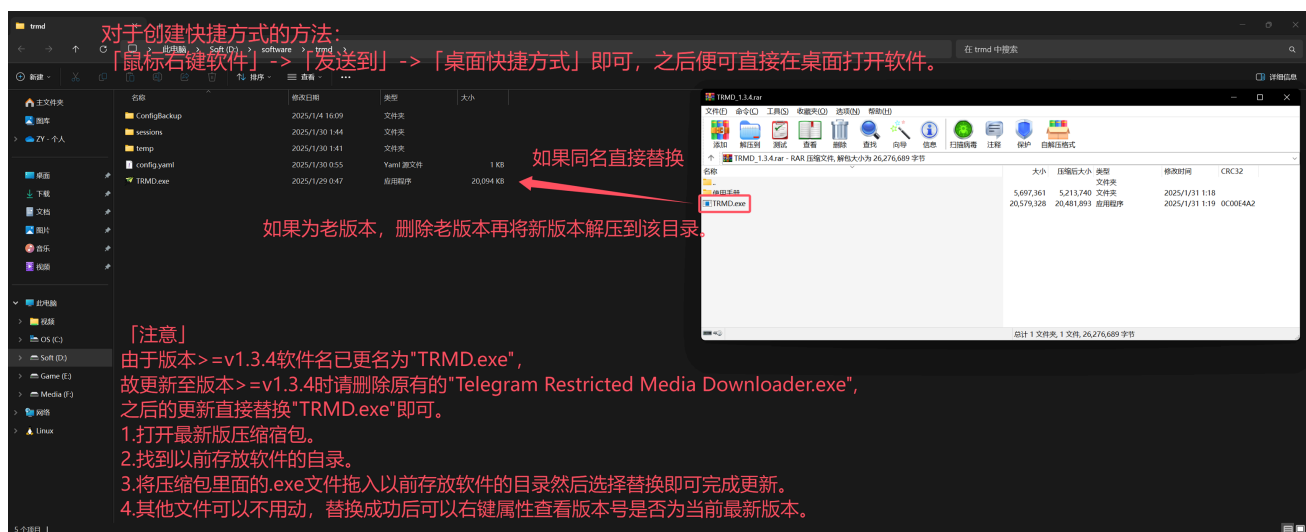
https://t.me/c/123456789/123?single

- **评论区单文件获取:** 需从评论区单独下载文件时，应采用复合参数格式：

https://t.me/c/123456789/123?single&comment=xx

- **参数互斥原则:**
  - 避免同时提交同一消息的完整版和单文件版链接。
  - 单文件模式与评论区下载模式（ ?comment ）不可混用。

## 2.5.软件更新教程:



## 3.0.在生产环境中运行:

推荐使用 `Python==3.13.2` 作为该项目环境(避免使用其他 `Python` 版本导致运行时出现报错)。

### 对于Windows用户:

需自行安装 `python` 与 `git` 并配置环境变量。

```
git clone https://github.com/Gentlesprite/Telegram_Restricted_Media_Downloader.git
cd Telegram_Restricted_Media_Downloader
python -m pip install --upgrade pip
pip install -r requirements.txt
python main.py
```

### 对于Linux用户:

克隆本项目并进入项目目录。

```
git clone https://github.com/Gentlesprite/Telegram_Restricted_Media_Downloader.git
cd Telegram_Restricted_Media_Downloader
```

更新 `pip` 版本(推荐)。

```
python3 -m pip install --upgrade pip
```

创建并使用**虚拟环境**(可选)。

```
python3 -m venv venv  
source venv/bin/activate
```

安装程序运行**所需依赖**并等待全部安装完成。

```
pip3 install -r requirements.txt
```

运行程序(到这一步就安装完成并运行了, 后面是注意事项)。

```
python3 main.py
```

## 注意事项:

**如果选择创建虚拟环境运行,在下次运行时也需要先激活虚拟环境。**

在项目目录下, **激活虚拟环境**后再运行程序。

```
source venv/bin/activate  
python3 main.py
```

如果提示**没有安装pip**使用如下命令进行安装:

```
sudo apt update  
sudo apt-get install python3-pip
```

## 关于更新:

---

在**项目目录**下打开终端使用如下命令拉取仓库当前的**最新版本**:

```
git pull
```

由于新版本可能使用了**新的依赖**, 使用 `git pull` 拉取后, 最好更新一下依赖(如果是**虚拟环境**请先激活再执行)。

pip3 install -r requirements.txt

## 4.0.(高阶用法)运行前设置命令行参数:

### Note

自版本  $\geq v1.8.3$  起：  
用户可在**运行前**通过**命令行参数**对**软件运行配置**进行**更多自定义设置**。

### ▼ 点击展开

**设置命令行运行参数**需先在**软件目录**打开**终端**，或**任意位置**打开**终端****进入软件目录**（经常使用**建议配置环境变量**）。

**目前支持的**命令行参数用法及解释如下表所示：

| 短参数 | 长参数       | 解释          |
|-----|-----------|-------------|
| -h  | --help    | 帮助          |
| -c  | --config  | 设置用户配置文件的路径 |
| -s  | --session | 设置会话文件的路径   |
| -t  | --temp    | 设置运行缓存的路径   |

**长参数与短参数最终结果一致。**

1. -h 、 --help 参数用法：

该参数用于获取使用帮助。

- 对于生产环境用户（**需要先完成前置步骤**"[3.0.在生产环境中运行](#)"）：

```
python3 main.py -h
```

```
python3 main.py --help
```

- 对于Windows用户：

```
TRMD.exe -h
```

```
TRMD.exe --help
```

- 对于Linux用户:

```
./TRMD -h
```

```
./TRMD --help
```

2. -c 、 --config 参数用法:

| 使用须知                                                                      |
|---------------------------------------------------------------------------|
| 1.该参数用于设置用户配置文件的路径。                                                       |
| 2.该参数旨在解决多个实例（多开）场景下，避免重复部署软件本体而设计的配置分离方案。                                |
| 3.该参数需指定一个符合 <a href="#">"2.3.配置文件说明(用户配置文件)"</a> 格式规范的文件，并且后缀名需为 .yaml 。 |
| 4.当指定的文件路径无效时，将使用软件默认设置。                                                  |

- 对于生产环境用户（需要先完成前置步骤["3.0.在生产环境中运行"](#)）：

以 Linux 系统为例（Winodws 系统同理），此处假设用户配置文件位于 /home/username/files/example.yaml 。

```
python3 main.py -c /home/username/files/example.yaml
```

```
python3 main.py --config /home/username/files/example.yaml
```

- 对于Windows用户:

此处假设用户配置文件位于 C:\files\example.yaml 。

```
TRMD.exe -c C:\files\example.yaml
```

```
TRMD.exe --config C:\files\example.yaml
```

- 对于Linux用户:

此处假设用户配置文件位于 `/home/username/files/example.yaml` 。

```
./TRMD -c /home/username/files/example.yaml

./TRMD --config /home/username/files/example.yaml
```

3. `-s`、`--session` 参数用法:

| 使用须知                                                                                           |
|------------------------------------------------------------------------------------------------|
| 1.该参数用于设置会话文件的路径。                                                                              |
| 2.软件会在用户登录成功时后，默认在运行目录下生成 <code>session</code> 文件夹，用于保存当前账号信息，以便后续快速登录。                        |
| 2.该参数旨在解决多账号登录场景下需 <b>手动重命名</b> 会话文件的问题，使用该参数用户可通过 <b>直接</b> 指定不同路径，选择对应账号进行登录。                |
| 3.该参数需指定一个 <b>文件夹</b> ，可包含已有的 <code>.session</code> 文件，指定为空文件夹或不存在时，登录后将在 <b>该路径自动生成</b> 会话文件。 |

- 对于生产环境用户（需要先完成前置步骤[3.0.在生产环境中运行](#)）：

以 Linux 系统为例（Winodws 系统同理），此处假设会话文件位于 `/home/username/files/session` 。

```
python3 main.py -s /home/username/files/session

python3 main.py --session /home/username/session
```

- 对于Windows用户:

此处假设会话文件位于 `C:\files\session` 。

```
TRMD.exe -s C:\files\session

TRMD.exe --session C:\files\session
```

- 对于Linux用户:

此处假设会话文件位于 `/home/username/files/session` 。

```
./TRMD -s /home/username/files/session
```

```
./TRMD --session /home/username/files/session
```

4. `-t`、`--temp` 参数用法:

| 使用须知                                                           |
|----------------------------------------------------------------|
| 1.该参数用于设置运行缓存的路径。                                              |
| 2.缓存即软件运行过程中，记录 <b>下载和上传</b> 信息的 <b>存储中转站</b> 。                |
| 3. <b>断点续传</b> 的实现同样离不开缓存机制的设计，用户可 <b>根据实际需求自定义缓存存放的路径</b> 。   |
| 4.该参数需指定一个 <b>文件夹</b> ，指定为空文件夹或不存在时，运行时将在 <b>该路径自动生成</b> 缓存文件。 |

- 对于生产环境用户（**需要先完成前置步骤**["3.0.在生产环境中运行"](#)）：

以 Linux 系统为例（Winodws 系统同理），此处假设缓存文件位于 `/home/username/files/temp` 。

```
python3 main.py -t /home/username/files/temp
```

```
python3 main.py --temp /home/username/temp
```

- 对于Windows用户:

此处假设缓存文件位于 `C:\files\temp` 。

```
TRMD.exe -t C:\files\temp
```

```
TRMD.exe --temp C:\files\temp
```

- 对于Linux用户:

此处假设缓存文件位于 `/home/username/files/temp` 。

```
./TRMD -t /home/username/files/temp
```

```
./TRMD --temp /home/username/files/temp
```

## 5.0.通过编译后运行:

---

**推荐**使用 `Python==3.13.2` 作为该项目环境(避免使用其他 `Python` 版本导致编译过程中或编译完成后出现报错)。

- 同"[3.0.在生产环境中运行](#)"前置步骤一致。
- 然后执行编译代码(建议使用**虚拟环境**，**避免**添加不必要的库，从而**减小**输出的文件大小)。

```
python build.py
```

## 6.0.联系作者:

---

Telegram:[@Gentlesprite](#)

邮箱:[Gentlesprites@outlook.com](mailto:Gentlesprites@outlook.com)

## 7.0.支持作者:

---





推荐使用支付宝



本伟 (\*\*字)

打开支付宝[扫一扫]

申请官方收钱码：拨打 95188-6

推荐使用微信支付



饿不死盘神 (\*\*字)



微信支付